ChatGPT

Executive Summary

Wazuh is a widely-used open-source **XDR/SIEM platform** for collecting and analyzing security events from endpoints, networks, and cloud systems【9†L212-L220】【23†L214-L222】. It uses agents and log collectors to ingest data, decoders and rules to normalize and detect threats, and an Elastic-based indexer/dashboard for search and visualization【17†L736-L744】【18†L866-L875】. However, studies have identified key limitations: the static rule-based engine in Wazuh produces high false-positive rates and lacks adaptive learning, leading to alert fatigue and limited scalability in high-volume environments【23†L214-L222】【23†L331-L339】.

This project will build an **Enhanced Wazuh-based SOC/XDR** system. We will start from the Wazuh baseline (manager, indexer, dashboard, agents) and **augment** it with new capabilities: intelligent alert triage, ML-based detection, threat-intel enrichment, automated response playbooks, attack-chain reconstruction, and a multi-layer correlation engine. The goal is to reduce false positives and mean time to detect/respond, improve visibility (including MITRE ATT&CK mapping), and enable security teams to focus on real threats. Key objectives include integrating Wazuh with network sensors (Zeek, Suricata), threat intelligence platforms (MISP, Wazuh CTI), and endpoint forensic tools (Velociraptor); adding a machine-learning tier to score and correlate events; and providing rich dashboards and APIs. The deliverable will be a complete SRS and design for this system, suitable for a research paper and implementation.

Project Scope and Objectives

- **Scope:** Enhance a Wazuh SIEM/XDR deployment with advanced correlation and intelligence. We will extend Wazuh's core (log collection, rules engine, indexer) by adding new modules (ML analytics, threat-intel, incident responder, custom dashboards) without altering Wazuh's GPLv2 code. The system will run on-prem or in the cloud, using containerization/infra-as-code as needed.

- **Objectives:**

- *Improved detection:* Integrate **ML models** to supplement Wazuh's static rules【8†L41-L49】【23†L214-L222】, enabling learning from historical data and catching anomalies.
- *Alert triage & prioritization:* Use ML and analytics to rank alerts by risk, reducing analyst fatigue.
- *Threat intel integration:* Automatically enrich alerts with known IOCs (via MISP or Wazuh CTI)【33†L10-L14】【44†L52-L60】.
- *Correlation and MITRE mapping:* Correlate events across layers (endpoint, network, cloud) and map all alerts to MITRE ATT&CK and compliance frameworks【17†L834-L842】.
- *Automated response:* Implement playbooks (block, quarantine, run commands) that can be triggered by high-confidence alerts【9†L282-L288】.
- *Attack-chain reconstruction:* Link related alerts into incidents, showing the sequence of tactics for investigations.
- *Custom rules and dashboards:* Provide tools for admins to define new detections and for analysts to build their own dashboards.

- *Scalable, resilient architecture:* Design for **high throughput** (hundreds of events/sec) and HA, leveraging Wazuh's clustering and indexing【18†L854-L863】【23†L248-L254】.
- *Multi-tenant support:* Enable separate workspaces/tenants with role-based access control (RBAC) for different SOC teams or customers.

Stakeholders and User Roles

- **Tier 1 SOC Analyst:** Monitors alerts on dashboards, does initial triage. Needs prioritized alerts and contextual data.
- **Tier 2/3 SOC Analyst:** Performs deeper investigation, uses correlation views, attaches MITRE techniques, escalates incidents. Requires tools for drilling into logs, reconstructing attack chains.
- **Threat Intel Analyst:** Manages threat feeds (MISP, CTI), configures intel enrichment, and vets new IOCs.
- **Incident Responder:** Uses the system to trigger containment playbooks, query endpoints (via Velociraptor), and document the response steps.
- **System Administrator:** Installs and maintains Wazuh services; configures log sources, rules, and integrations; ensures availability.
- **DevOps/CI-CD Engineer:** Maintains the deployment pipeline (Ansible/Kubernetes); automates updates to rules and software.
- **Compliance Officer (audit role):** Ensures data retention and access controls meet standards.

System Overview and Architecture

The enhanced SOC/XDR system builds on the **Wazuh architecture**: agents collect logs/events from endpoints (Windows/Linux/macOS), which are sent to the Wazuh Manager. The manager analyzes events via decoders and rules, generating alerts. Alerts and event data are forwarded to the Wazuh Indexer (Elasticsearch) and exposed in the Wazuh Dashboard【17†L736-L744】【18†L866-L875】.

Our **extended architecture** (Figure below) adds multiple ingestion sources (network and cloud telemetry, external tools), an ML/AI analytics layer, and integration modules for threat intelligence and automated response. Key components include:

- **Data Sources:** Endpoints/servers (Wazuh agents), network sensors (Zeek for DNS/HTTP logs; Suricata IDS for network anomalies), cloud logs (AWS/Azure events), and EDR (Velociraptor).
- **Wazuh Manager:** Central analysis engine (log normalization, ruleset, MITRE mapping)【17†L809-L818】【17†L834-L842】. Modified to emit events to the ML engine.
- **Wazuh Indexer (Elasticsearch):** Stores all alerts and raw events in indices (`wazuh-alerts-*`, `wazuh-archives-*`, etc.)【18†L866-L875】. Provides search and retention.
- **Wazuh Dashboard:** Kibana-based UI with out-of-the-box security dashboards and custom ones. Extended to support multi-tenant (workspaces) and new visualizations.
- **ML/AI Engine:** A new processing layer that consumes events (via Elasticsearch or Kafka) and computes anomaly scores or classifications. Results are fed back as enriched alerts. (E.g. Random Forest or DBSCAN as in Chamkar et al.【8†L41-L49】【23†L276-L284】.)
- **Threat Intelligence Integrator:** Module that queries CTI sources (MISP, open CTI, VirusTotal, etc.) for IOCs. E.g. Wazuh's Integrator framework can run a custom Python script to ask a MISP API when

a new file hash alert appears【33†L10-L14】【33†L66-L74】. Also fetches CVE data from the Wazuh CTI service【44†L52-L60】.

- **Automated Response Engine:** Uses Wazuh's existing **active response** framework【9†L282-L288】 and new playbooks (e.g. in Python/Ansible) to take actions (block IPs, kill processes) when certain alerts (high severity) are raised.
- **APIs and Dashboards:** RESTful APIs for querying data; role-based dashboards.

```
graph LR
  subgraph DataSources
    A[Endpoints/Servers] -->|Syslog/Agent| WM[Wazuh Manager]
    B[Network (Zeek IDS)] -->|Filebeat/Log| WM
    C[Suricata IDS] -->|EVE JSON| WM
    D[Velociraptor EDR] -->|Logs/API| WM
    E[Cloud/Apps] -->|Cloud APIs| WM
    F[MISP/Threat Feeds] -->|HTTP/API| WM
  end
  subgraph WazuhPlatform
    WM --> WI[Wazuh Indexer (Elastic)]
    WM --> UI[Wazuh Dashboard]
    WM --> ML[ML/AI Engine]
    ML --> WM
  end
  UI --> Analyst[Security Analyst]
  Analyst --- WM
```

Figure: High-level architecture. Data (logs/alerts) flow into Wazuh Manager, which sends alerts to the Elasticsearch indexer. A new ML/AI Engine enriches detections, and external feeds/EDR are integrated. Analysts use the dashboard and APIs to interact.

Functional Requirements

1. **Data Ingestion:** The system must ingest logs/events from diverse sources: Wazuh agents (syslog, JSON logs), cloud APIs (AWS CloudTrail, Azure Monitor), network tools (Zeek, Suricata), and third-party tools (Velociraptor, Sigma-converted alerts). Wazuh's flexible log collection supports syslog and API inputs【17†L736-L744】.
2. **Normalization (Decoders):** All incoming data must be parsed into a common schema. Wazuh decoders transform raw logs (Windows event XML, Linux syslog, Zeek TSV, Suricata JSON) into structured JSON fields【17†L809-L818】. Custom decoders should be defined for new formats (e.g. Zeek, Velociraptor).

3. **Correlation and Rule Engine:** The core Wazuh rules engine applies pattern-matching rules to normalized events【17†L823-L831】. Requirements:

4. *Rule chaining:* Support "if_sid" style rule correlation (Alert B fires only if Alert A occurred) to detect multi-step attacks.

5. *Thresholds and frequency:* Rules can use frequency count (e.g. X failed logins in Y time).
6. *Custom rule management:* Administrators can add/edit rules through UI or files, with immediate effect.
7. **Alert Triage & Prioritization:** Implement a scoring mechanism so that each alert is enriched with a "confidence" or risk score. ML models (e.g. Random Forest, clustering) will label alerts as true/false positives【8†L41-L49】【23†L331-L339】. The system must allow configuring thresholding (e.g. only escalate level ≥N) and automatic tagging of low-confidence alerts as "review later."
8. **MITRE ATT&CK Mapping:** Each alert should include associated MITRE ATT&CK technique IDs and tactics. Wazuh's built-in rules can include `<mitre>` tags【17†L834-L842】. New mappings (including links from Sigma or custom rules) must be supported.

9. **Threat Intelligence Enrichment:** For relevant events (file creation, network connection, login, etc.), the system will query external intelligence for IOCs:

10. Query Wazuh CTI API for CVE info and vulnerability context【44†L52-L60】.

11. On file hash alerts (ID 554), call a MISP API to check hash; generate a high-severity alert on match 【33†L10-L14】.
12. Ingest feeds of malicious IPs/domains via STIX/TAXII or CSV, and flag events matching them.
13. **Automated Response Playbooks:** Define response actions tied to alerts. E.g., on malware detection, quarantine host or block IP via firewall. The system must support scripting of responses (via Wazuh's `active response` or external orchestration). Incident responders can also manually execute playbooks.
14. **Attack-Chain Reconstruction:** When multiple alerts relate to an incident, the system should group them into an "incident" record. The UI must display an incident timeline (from initial foothold to exfiltration). This requires linking alerts by host/user/session.
15. **Custom Detection Rules:** Provide an interface/API to create and test new detection rules (in Wazuh's XML format or via a Sigma converter【30†L73-L80】). Acceptance criteria include: new rule triggers expected alerts with no syntax errors.
16. **Dashboards & Visualization:** Out-of-the-box dashboards must display key SOC metrics (top alerts, MITRE tactics, system health). Users can create custom dashboards via the Wazuh Dashboard (based on Kibana)【18†L902-L913】. Multi-layer drill-downs (from overview to individual event) are required.
17. **Multi-Tenant Support:** The system must logically isolate data by team or customer. Each tenant sees only their agents and alerts. Wazuh can enable multi-tenancy via separate index/workspace configurations (e.g. Kibana spaces)【18†L902-L913】.
18. **APIs:** All core data (alerts, incidents, rules) must be accessible via REST APIs. Wazuh's Server API already provides querying (WQL)【17†L809-L818】; we extend it as needed for incident management and integrations.
19. **Logging and Audit Trail:** Every configuration change (rule edits, user role changes) and system action (alert forwarding, response scripts) must be logged. The system should audit who did what and when, for compliance.

Non-Functional Requirements

- **Scalability:** The system must scale to large deployments (e.g. 10,000+ agents, 1,000+ EPS). Wazuh's indexer is distributed and sharded【18†L861-L870】, so add nodes as needed. Design should allow horizontal scaling of manager and indexer components.
- **Performance & Latency:** The enhanced pipeline must process events in near-real-time. Targets (based on Chamkar et al.【8†L49-L53】【23†L290-L298】): end-to-end detection latency ≤100ms per event, 95% of events under 500ms, and linear throughput up to at least 500 eps. ML inference should be optimized (e.g. 45ms avg in tests【8†L49-L53】).
- **Availability & Fault Tolerance:** Wazuh Manager and Indexer should be deployed in cluster mode. The indexer uses replication across nodes【18†L861-L870】. Define RPO/RTO for major failures (e.g. <1h RPO, <4h RTO). Use redundant network paths.
- **Security:** All internal communications (agents↔manager, manager↔indexer) use TLS with mutual auth. The dashboard/API require authentication and authorization (Wazuh supports RBAC)【36†L739-L743】. Sensitive data (PII) must be masked or encrypted at rest as needed.
- **Data Retention & Compliance:** Raw events (wazuh-archives) may require long-term storage. Provide configurable retention (e.g. hot vs cold storage) and GDPR/PCI controls as needed. Logs should be immutable once written.
- **Maintainability:** The system should support Infrastructure-as-Code (Ansible roles【18†L854-L863】, Docker/K8s manifests) so that it can be replicated and upgraded easily. Use CI/CD pipelines for code/rule updates.
- **Observability:** Collect metrics on component health (queue lengths, CPU, memory) and define alerts on them. Integrate with Prometheus/Grafana or built-in Kibana monitoring.

Data Model and Schemas

We define core entities as follows (examples):

Event	Description
`event_id` (UUID)	Unique ID for the event.
`timestamp`	UTC time of the event occurrence.
`source_host`	Hostname/IP where event originated.
`source_type`	Type (e.g. "WindowsSecurityLog", "Zeek.conn.log", "Suricata.eve").
`raw_message`	Original raw log text (or JSON blob).
`normalized_fields`	Parsed fields (e.g. `user`, `file_hash`, `dst_ip`, `process_name`, etc.).
`agent_id`	Wazuh agent identifier (if any).
`severity`	Initial severity or priority.

Alert	Description
`alert_id` (UUID)	Unique ID for the alert.
`event_id` (ref)	Reference to the triggering event.
`rule_id`	ID of the Wazuh rule that fired.
`severity_level`	Numeric level (1–16) as per rule.
`description`	Human-readable description of the alert.
`tags`	List of tags/groups (e.g. ["malware","GDPR"]).
`mitre_techniques`	List of MITRE ATT&CK IDs (e.g. T1078).
`analysis_notes`	Analyst comments or context (optional).
`investigation_status`	E.g. “New”, “Investigating”, “Resolved”.
`enriched_data`	Additional fields (e.g. `file_hash_match: true`, `cve_ids: ["CVE-2025-0411"]`).

Incident	Description
`incident_id` (UUID)	Unique identifier for the incident.
`start_time` / `end_time`	Time window of the incident.
`severity_score`	Aggregated risk score (e.g. max of contained alerts).
`summary`	Brief description (e.g. “Brute-force SSH with data exfiltration”).
`alert_ids` (list of IDs)	All associated alerts.
`status`	Status (Open, In Progress, Closed).
`assigned_to`	Analyst/team handling it.
`resolution`	How it was resolved (e.g. “Blocked user account, patched software”).

ThreatIntelIndicator	Description
`ioc_id` (UUID)	Unique ID for the indicator.
`type`	“IP”, “Domain”, “FileHash”, “URL”, etc.
`value`	The indicator value (e.g. “192.168.1.100”).
`source`	Origin (e.g. “MISP feed”, “VirusTotal”, “Wazuh-CTI”).
`first_seen` / `last_seen`	Timestamps when IOC observed.
`threat_type`	e.g. “Malware”, “Phishing”, “C2”.
`confidence`	Confidence level from source (e.g. “High”, “Unknown”).

Tables: Sample schema for events, alerts, incidents, and threat intelligence indicators. These reflect Wazuh's JSON structure (alerts are JSON docs in Elasticsearch【18†L866-L875】). Wazuh stores raw events and alerts in indices like `wazuh-archives-*` *and* `wazuh-alerts-*` *【18†L866-L875】.*

Integration Points

Component	Integration Method	Data/Protocol Format
Zeek (Network IDS)	Zeek runs on a host; its logs (conn.log, dns.log, etc.) are converted to JSON via Filebeat or custom script【25†L90-L99】. Wazuh ingest reads JSON.	JSON (Zeek fields mapped to standard names)【25†L90-L99】.
Suricata (NIDS)	Suricata's EVE JSON logs are forwarded to Wazuh (via Filebeat or directly).	JSON (EVE schema) – fields like src_ip, alert.signature, etc.
Velociraptor (EDR)	Velociraptor client on endpoints can send forensic data (process lists, network connections) to a central server. Wazuh agent can tail Velociraptor logs or use API scripts.	JSON or CSV logs; Wazuh can parse Velociraptor output as log events.
MISP (Threat Intel)	Wazuh "Integrator" module calls MISP API over HTTPS when a matching event (e.g. file hash) occurs【33†L66-L74】. Results parsed by a Python script.	HTTPS JSON API (queries: get attributes; returns JSON of IOCs)【33†L66-L74】.
Wazuh CTI (Vuln Intel)	Wazuh calls CTI REST API to fetch CVE data for detected vulnerabilities【44†L52-L60】.	JSON (CVE data in CVE JSON 5.0 format)【44†L91-L100】.
Sigma Rules	Sigma rules are converted to Wazuh rule XML using a converter tool (e.g. Chainsaw or sigma_to_wazuh)【30†L73-L80】. The generated rules are loaded into Wazuh.	Conversion result: Wazuh XML rule format; ingestion as normal rules.
Splunk	(Optional) Wazuh can forward alerts to Splunk via syslog or HTTP Event Collector【36†L734-L743】【36†L798-L807】. Conversely, Splunk logs could be forwarded into Wazuh agents or via syslog.	Syslog (UDP 514) or HEC; raw text or JSON from Splunk.
Threat Feeds (Open)	Periodic pulls or STIX/TAXII clients ingest feeds (e.g. malware IP lists, Tor exit nodes). Wazuh can read these as local CSV lists (CDB lists) or via API.	CSV/JSON feeds; CDB list format for Wazuh rules【17†L823-L831】.

Table: Integration points with other tools. Zeek and Suricata feed network event logs; Velociraptor provides endpoint forensic logs; MISP and Wazuh CTI supply external intelligence; Sigma rules are converted to Wazuh rules. Wazuh agents and indexers communicate via filebeat/syslog/HTTP.

Deployment and Environment Requirements

- **Deployment Mode:** Support on-premises (bare-metal/VM) or cloud (AWS/Azure) or hybrid. Use containerization (Docker, Kubernetes) for manager, indexer, and dashboard to ease scaling. A reference architecture is Wazuh's Docker or Kubernetes deployment (see Wazuh docs).
- **Infrastructure as Code:** All components must be deployable via Ansible/Puppet scripts【18†L854-L863】 or Helm charts. For example, Wazuh provides Ansible roles for manager, indexer, and agent【18†L854-L863】. The CI/CD pipeline will trigger deployments.
- **High Availability:** Use multiple manager and indexer nodes with load balancers. For example, an active-passive Wazuh cluster or multiple indexer nodes with replication.
- **Access:** Web UIs and APIs served over HTTPS with certificates. Network ports secured (e.g. 1514/1515 for agents, 514 for syslog if used).
- **DevOps (CI/CD):**
- Maintain Wazuh rules and configurations in Git.
- Automatic testing (syntactic validation of XML rules, linter for scripts).
- Automated builds/deployments when code is merged (e.g. Jenkins, GitHub Actions).

Testing Plan

- **Unit Testing:** Develop unit tests for new parsers, ML modules, and scripts. For example, test ML classifiers on labeled datasets; test integrator scripts with mocked API responses.
- **Integration Testing:** Simulate end-to-end flows: e.g., generate logs on an endpoint, verify they become alerts in the dashboard. Use tools like OVAL, Atomic Red Team, or Caldera to replay attacks.
- **Performance Testing:** Stress-test ingestion and alert pipeline. Tools like Apache JMeter or in-house scripts will push thousands of events/sec to Wazuh; measure CPU/memory, queue lengths, and latency. Targets: as noted, <100ms latency up to ~500 eps【8†L49-L53】【23†L290-L298】.
- **Security Testing:** Conduct vulnerability scans on the deployed system. Verify encryption (test MITM attempts) and perform RBAC checks.
- **Red-Team/Attack Simulations:** Plan adversary emulations covering typical kill-chain (phishing → C2 → exfiltration). Document if alerts fired and how they propagate through system. Metrics: detection rate (true positives / actual attacks), false positive rate (benign events flagged), MTTD (mean time to detect), MTTR (mean time to respond). For reference, industry SOC KPIs like MTTD/MTTR should be measured【40†L0-L9】.
- **Evaluation Metrics:** Use metrics from Chamkar et al.【8†L45-L54】 as benchmarks: classification accuracy (~97%), FP rate (<0.1), processing latency (avg ~45ms). We will also report detection precision/recall, and analyst workload reduction (via surveys or time studies).

Implementation Roadmap

A high-level phased plan (assumes a team of 3–5 FTEs over ~12 months). Major milestones:

```
gantt
    dateFormat  YYYY-MM-DD
    title Implementation Timeline (Example)
```

```
    section Phase 1: Research & Design
    Baseline Setup             :done,    a1, 2026-09-01, 14d
    Requirements Analysis      :active,  a2, 2026-09-15, 10d
    Architecture Design        :         a3, 2026-09-25, 14d
    section Phase 2: Core Dev
    Log Ingestion Module       :         a4, after a3, 2026-10-10, 21d
    Rule Engine Extensions     :         a5, after a4, 2026-10-31, 21d
    ML/Alert Triage Engine     :         a6, after a5, 2026-11-21, 30d
    Threat Intel Integration   :         a7, after a6, 2026-12-21, 21d
    section Phase 3: Testing & Refinement
    Unit/Integration Testing   :         a8, after a7, 2027-01-11, 14d
    Performance Tuning         :         a9, after a8, 2027-01-25, 14d
    Red-Team Simulations       :         a10, after a9, 2027-02-08, 14d
    section Phase 4: Deployment & Documentation
    Final Deployment           :         a11, after a10, 2027-02-22, 10d
    Documentation & Training   :         a12, after a11, 2027-03-04, 7d
```

Figure: Sample Gantt chart showing phases. Each task has estimated start and duration (in days). "after X" links start times.

Milestones:

- *Phase 1:* Set up base Wazuh (manager+indexer) and gather requirements.
- *Phase 2:* Develop ingestion (Zeek/Suricata logs, cloud logs), extend rules, integrate ML triage, CTI integrations.
- *Phase 3:* Testing in lab: unit tests, scalability tests, red-team. Adjust based on results.
- *Phase 4:* Final deployment (production environment), write research paper and user documentation.

Effort Estimates: Each bullet above assumes parallel streams. Total ~6 months dev + 3 months testing + 3 months buffer.

Risk Analysis & Mitigation:

- *False Positives Too High:* If ML models or rules still flag many false alerts, adjust models or tighten thresholds. Mitigation: iterative tuning with labelled data; allow manual override.
- *Performance Bottlenecks:* Elastic cluster might lag under load. Mitigation: add indexer nodes/shards; use Kafka buffering if needed (Wazuh supports Kafka).
- *Integration Complexity:* APIs (MISP, cloud) might change or fail. Mitigation: build abstraction layers and fallbacks.
- *Data Privacy:* Handling sensitive logs could breach privacy. Mitigation: implement field masking (e.g. PII removal plugins) and strict access control.
- *Team/Time Overruns:* The project is large. Mitigation: focus on MVP features first (core ingestion, basic rules, MITRE) before fancy extras.

Research Gaps and Future Work

While our SRS outlines many enhancements, further research areas include:

- **Advanced ML/AI:** Explore deep learning or unsupervised methods for anomaly detection beyond RF/DBSCAN【23†L276-L284】. Research Explainable AI (XAI) to help analysts understand ML-driven alerts.
- **User Behavior Analytics (UBA):** Add models that profile user/network baselines and flag deviations (insider threats).
- **Automated Forensics:** Integrate forensic data collection (with Velociraptor or OSQuery) triggered by alerts, to capture volatile state.
- **Cross-Platform Correlation:** Research linking detections across SIEM and EDR in a knowledge graph for root-cause analysis.
- **Security Data Pipelines:** Evaluate using a security data lake (e.g. Splunk/Snowflake) alongside Wazuh for big-data queries as in Gartner's "Security Data Pipelines"【41†L20-L23】.

Prioritized Feature List and Acceptance Criteria

Feature	Stage (MVP/v1/v2)	Acceptance Criteria
Log Ingestion & Parsing	MVP (core)	Ingest logs from Windows/Linux/macOS agents and syslog; all must appear as events in Wazuh indices【17†L736-L744】.
Rule-based Alerts	MVP	Built-in Wazuh rules fire correctly; MITRE IDs appear on alerts【17†L834-L842】.
Custom Rule Creation	MVP	Admins can add a new rule file via UI; it triggers expected alerts without errors.
Dashboard Views	MVP	Default security dashboards show alerts (e.g. Failed logins, Malware) as in Wazuh out-of-box【18†L908-L916】.
Alert Forwarding (Syslog/API)	MVP	Alerts can be forwarded to an external syslog receiver (test OK); API query returns alert list.
Threat Intelligence Query	v1	On a file-hash alert, MISP integration returns a match and a new alert (ID 100802) fires【33†L10-L14】.
Wazuh CTI Integration	v1	Vulnerability alerts in Wazuh link to CTI info; CVE details are accessible via API【44†L130-L139】.
Alert Prioritization (ML)	v1	ML engine labels ≥90% of true attacks correctly (precision/recall) and reduces FP by ≥50% (benchmarked).
Automated Response Playbook	v1	A designated alert (e.g. malware) automatically triggers a block-response script. Playbook executes without errors.

Feature	Stage (MVP/v1/v2)	Acceptance Criteria
Multi-source Correlation	v1	Events from Zeek/Suricata that are related to a host are correlated into one incident in UI.
Multi-Tenant RBAC	v2	Two user accounts (Tenant A, Tenant B) see only their own agents/alerts when switching workspaces.
Advanced Analytics (UEBA)	v2	Anomaly model trained on normal traffic flags at least one real-world outlier in testing, with <5% false alarms.
Incident Management UI	v2	Analyst can view/edit an “Incident” grouping alerts, add notes, and mark it resolved. Status changes recorded.

Table: Feature rollout plan. MVP features are core ingestion, rules, and UI. v1 adds ML and intel integrations. v2 includes advanced analytics and operations. Each acceptance criterion is measurable (e.g. “generates expected alert”, “<X% false positives”).

Conclusion: This SRS lays out a comprehensive plan to take a Wazuh deployment well beyond its defaults, addressing known gaps (high false-positives, lack of automation【23†L214-L222】【23†L331-L339】) with cutting-edge enhancements. By following this specification, the project will yield a robust, next-generation open-source SOC/XDR platform, suitable for publication as a research work and for use in enterprise environments.

Sources: Official Wazuh documentation and GitHub (architecture, decoders/rules, indices)【17†L736-L744】【18†L866-L875】; academic papers on Wazuh ML enhancements【8†L41-L49】【23†L214-L222】; Wazuh engineering blogs on Zeek and CTI integration【25†L59-L68】【44†L52-L60】; threat intel (MISP) tutorial【33†L10-L14】【33†L66-L74】; and other open-source SIEM literature. All statements above are grounded in these sources.